Fifty sleeps of 3.2 ms took 756 ms, and that is why I deleted my RTP pacer

Correction, 30 October 2026. An earlier version of this essay said aiortc cuts H.264 keyframes at fixed 1200-byte offsets. That was my error: aiortc splits on NAL start codes, and the section is rewritten below. Also corrected: where aiortc runs the encode, the keyframe size behind the 750 ms lockout, and two before-the-pacer baselines that had no measurement behind them.

I built a self-hosted remote desktop for Windows that streams the screen to a browser tab over WebRTC. It works. The part worth writing down is a fix I was confident about, shipped, measured, and then removed.

The symptom

Round-trip time measured in the browser swung between 7 ms and 7000 ms. The path was Tailscale direct, 2 ms raw RTT, no packet loss. Server-side logs insisted capture and delivery were both a steady 30 fps.

Two instruments disagreeing by three orders of magnitude usually means one of them is measuring something other than what its name says.

The hypothesis that was right

aiortc ships no RTP send pacer. The encode itself runs on a worker thread, but the packets of each frame come back to the same asyncio event loop that runs ICE, DTLS and the input data channel, and leave there in a tight `for` loop of synchronous sends with nothing between them. That is confirmed in aiortc discussion #965 and visible by reading `rtcrtpsender.py`.

A keyframe at 3 Mbps is around 50 RTP packets at the GOP this host ships. Emitting them back to back inside the event loop can starve ICE heartbeats for the duration. Starved heartbeats explain a 7-second RTT reading much better than a 2 ms network does.

So far so good. The diagnosis held up.

The fix that made it worse

I wrote a leaky-bucket pacer. One `asyncio.sleep(9600 / bitrate)` between packets, which at 3 Mbps is about 3.2 ms. Standard approach, roughly what libwebrtc does.

By the notes I left in the code at the time, delivered frame rate collapsed to 2 or 3 fps, keyframes stalled for most of a second, and per-frame `pipeline_gap` sat somewhere between 222 and 504 ms. No log from those runs survives.

I had made the thing I was fixing worse.

What was actually happening

Windows keeps its default system timer granularity near 15.6 ms. Every timer-backed wait rounds up to that tick. `asyncio.sleep(0.0032)` does not sleep for 3.2 ms on stock Windows. It sleeps for about 15.

I timed it directly rather than reasoning about it:

```
fifty consecutive asyncio.sleep(3.2ms) calls -> 756.7 ms of wall clock
```

Fifty sleeps that should have cost 160 ms cost 756.7 ms. A 50-packet keyframe was therefore locked out of the send path for roughly three quarters of a second, every single time a keyframe went out.

The pacer was solving a congested-WAN problem that a 2 ms tailnet link does not have, and paying for it with a Windows timer bill I had not measured.

What replaced it

`install_send_pacer()` is now a documented no-op. The one thing it still does is this:

```
winmm.timeBeginPeriod(1)
```

That single call drops system timer granularity from 15.6 ms to 1 ms process-wide. It does nothing for my deleted pacer, because the pacer is gone. What it does is make every *other* short sleep inside aiortc accurate: RTCP timing, retransmit scheduling, every internal wait that was silently rounding up to 15 ms and that I had never thought to look at.

The keyframe burst problem got solved the boring way instead, by raising GOP from 60 to 120 frames. Half as many keyframes, half as many bursts.

The other bug, which was not a bug

While in there I convinced myself that aiortc fragments the H.264 bitstream at fixed 1200-byte offsets, so a keyframe carrying SPS, PPS and an IDR slice would be cut mid-NAL and arrive corrupt. I wrote `_nal_aware_split_bitstream` to cut on Annex-B start codes first and leave MTU fragmentation to FU-A.

That belief was wrong. aiortc has split on start codes since at least version 0.9.10 in 2018, and so does the 1.14.0 this project runs. Fed a keyframe of a 25-byte SPS, a 5-byte PPS and a 120,000-byte IDR slice, aiortc's own splitter keeps all three NALs whole, and they rebuild byte for byte from its 94 RTP packets. My splitter produces the same three pieces. The patch changes nothing.

It is the pacer lesson again, one size smaller: a fix written from a confident reading, never checked against the code it was fixing.

I also swapped libx264 for `h264_nvenc` with `preset=p1`, `tune=ull`, `rc-lookahead=0`, `surfaces=2`. Measured in isolation, that encodes a 1080p frame in 10.8 ms.

Where it actually stands

The last measured session captured at 30.0 fps and delivered 8.6 fps to the browser, with 42.5 ms spent inside `recv()` and a 70.4 ms pipeline gap per frame. It is watchable, input feels close to immediate at lower resolutions, and 1080p30 is not there. The remaining bottleneck is the Python RTP send path, which is why there is now a second host built on GStreamer `webrtcbin` with a zero-copy D3D11 capture and encode path.

There is also no authentication of any kind. Anyone who reaches the port gets keyboard and mouse. I run it inside a tailnet and that tailnet is the entire security model.

The thing I would say in an interview

The correct diagnosis and the correct fix are not the same artifact, and being right about the first one bought me nothing.

I was right that aiortc has no pacer. I was right that unpaced bursts starve the event loop. The pacer was still the wrong move, and I would not have known that from reading more about WebRTC congestion control, because the failure was not in WebRTC at all. It was in what `sleep` means on the operating system I happened to be running.

The measurement that mattered was never on the network. It was on the clock the sleep call was using.

Fabi-SPL · github.com/Fabi-SPL/essays
Code, including the deleted pacer left in as a documented no-op with the reasoning attached: https://github.com/Fabi-SPL/remote-desktop